AlgoFlow Project Report

Personal DSA revision and solution history

Submitted by Krish Sondagar | 15 October 2026

Source repository https://github.com/KRISH141006/AlgoFlow

1 Project overview

AlgoFlow is a local web application that stores accepted DSA solutions, schedules revision, and records how a solution changes over time. This report describes the implemented demonstration release, its system design, database schema, workflows, screenshots, and verified test results.

Problem statement

Coding platforms record submissions, but the student still needs a practical way to revisit an approach, locate an earlier solution, and distinguish a saved attempt from an accepted submission. Local code files and topic notes do not provide a consistent revision schedule. Repeated imports can also create duplicate records if problem and submission identity are not handled explicitly.

Objectives

- Consolidate exported LeetCode and GeeksforGeeks solutions in a searchable library.
- Preserve exact source code, problem descriptions, topic tags, and solution assessments.
- Schedule explicit problem and theory reviews without altering existing revision history.
- Provide persistent storage, duplicate protection, and complete backup and additive restore.

Key features

The application provides four views: Today, Library, Problem Detail, and Progress. Users can import JSON, add problems and topics, view stored solutions, save a new version, complete a due review, and export or restore a backup. Acceptance status and review status are separate: an accepted problem can still be pending revision.

Implementation scope

This is a single-user application built with FastAPI and React. It stores data locally in SQLite and does not implement login, account registration, roles, cloud hosting, or live browser-extension capture. The demonstration implementation was prepared with AI assistance; the imported solution collection and handwritten Universal Sync extension are existing student work.

2 System design

The frontend and backend are served from one local address. HTTP requests enter FastAPI routes, pass through request validation and service functions, and read or update the selected SQLite database. The original tracker is read only during preparation.

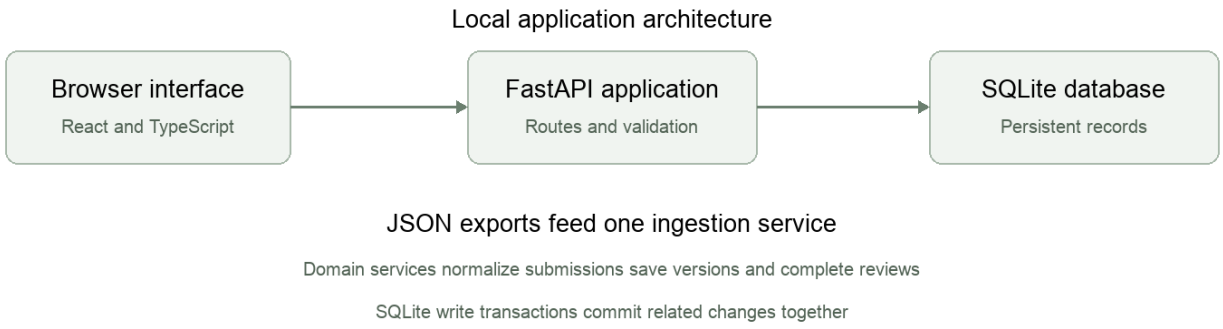


Figure 1 Local application architecture

Component responsibilities

Component	Responsibility
React interface	Displays queues, filters, descriptions, code snapshots, forms, and progress.
FastAPI routes	Expose /api/v1 endpoints and translate HTTP requests into application operations.
Pydantic schemas	Validate required values, lengths, difficulty, problem URLs, and expected review stage.
Domain services	Normalize platform exports, deduplicate submissions, create snapshots, and apply revision rules.
SQLite access	Use parameterized queries, foreign keys, repeatable migrations, and write transactions.
Backup service	Export all five tables and restore with validation and parent ID remapping.

Data flow

A JSON file is read and parsed, then each submission is normalized into a common internal format. Valid records are stored together in a transaction. The backend returns counts of created, updated, skipped, and invalid records. The interface refreshes its library and statistics from the database after the operation.

Personal and demonstration data

The preparation script creates a consistent snapshot of the original tracker, migrates a working personal database, and creates a separate demo database. Presentation actions affect only the selected database. Refreshing the demo first saves a backup of its current state.

3 Database schema for problems and topics

SQLite stores five related tables. Integer primary keys identify records inside AlgoFlow; platform IDs and URL slugs identify external problems. JSON tags are stored as TEXT and decoded for API responses.

Problems table

Fields	SQLite type	Purpose
id	INTEGER primary key	Local problem identifier.
name difficulty link	TEXT	Display label, Easy/Medium/Hard classification, and reference URL.
date_added	TEXT	Original tracker addition date or administrative creation date.
stage status next_review_date	INTEGER and TEXT	Revision stage 0 to 4, pending/mastered state, and due date.
platform slug platform_id	TEXT	Platform identity and external metadata.
topic_tags company_tags	TEXT containing JSON	Topic and company labels.
problem_html	TEXT	Sanitized question description for local reading.
imported_at solved_at	TEXT nullable	Import timestamp; solve timestamp is left unavailable when absent.
historical_import	INTEGER	Distinguishes historical collection imports from manual additions for activity counting.
solve_state	TEXT	Accepted or unverified solution state.

Topics table

Fields	SQLite type	Purpose
id	INTEGER primary key	Local theory topic identifier.
name category notes link	TEXT	Concept name, grouping, personal explanation, and reference.
date_added	TEXT	Date the topic was added.
stage status next_review_date	INTEGER and TEXT	Theory revision progression and due date.

Identity and migration rules

Problems are matched by platform and canonical URL slug. Existing labels, IDs, stages, and dates are retained when an import enriches a tracker record. Schema upgrades add columns and tables rather than deleting the original records. The problems identity index supports lookup; ingestion serializes writes to prevent competing duplicate inserts.

4 Database schema for versions and reviews

Problem versions table

Fields	SQLite type	Purpose
id problem_id	INTEGER	Snapshot identifier and foreign key to problems.
version_number	INTEGER	Sequential number within one problem.
commit_message code language	TEXT	Explanation, exact submitted code, and programming language.
what_is_good what_is_bad	TEXT	Self-assessment of the approach and improvements.
time_complexity space_complexity	TEXT	Optional time and space complexity notes.
created_at accepted source	TEXT and INTEGER	Snapshot creation time, acceptance flag, and import/manual origin.
submission_id	TEXT nullable	External submission identity when available.
fingerprint delivery_key	TEXT	Code digest and repeat-delivery identity.

The database enforces unique (problem_id, version_number) and (problem_id, delivery_key) pairs. A foreign key associates every snapshot with a problem. There is no endpoint that edits a saved code snapshot.

Review history tables

Field	Type	Meaning
id	INTEGER primary key	Review event identifier.
problem_id or topic_id	INTEGER foreign key	Links to problems or topics respectively.
review_date	TEXT	Completion date in Asia/Kolkata.
stage_completed	INTEGER	Stage reached after completing the review.
was_on_time	INTEGER	1 when completed on its scheduled day, otherwise 0.

review_history belongs to problems; topic_review_history belongs to topics. One problem can have many versions and reviews. One theory topic can have many reviews. Versions and reviews are independent records, so saving code does not silently complete a review.

Consistency guarantees

Every connection enables foreign keys. Related writes run in a transaction using BEGIN IMMEDIATE. Unexpected database errors roll back the operation. Backup restoration validates parent references and remaps identifiers when the receiving database already contains different records.

5 Implementation details

Submission import

The normalizer accepts a single object or an array. LeetCode exports use userCode and problem_html; older GFG exports can use submission.code and problem_question. Names, platform identity, slugs, difficulty, code, language, and tag lists are validated. Ambiguous platform information is reported rather than guessed.

A supplied submission ID is used for duplicate detection. Without one, the backend hashes the language and exact code within the problem. Distinct submission IDs preserve separate submissions. Deliberate manual commits may keep identical code with a different assessment.

Review completion

State after action	Next review	Task
Added at stage 0	Tomorrow	First recall
Stage 1 completed	7 days later	Second recall
Stage 2 completed	30 days later	Re-solve
Stage 3 completed	180 days later	Long-term re-solve
Stage 4 completed	None	Mastered

A completion request includes expected_stage. The service checks that the item exists, is due, is not mastered, and still matches that stage. It logs the review and advances exactly one stage in the same transaction. A repeated or stale request receives a conflict response.

Forms and validation

React forms support problem entry, theory notes, solution versions, and file uploads. Required fields and URL controls provide browser feedback; Pydantic performs backend validation. Missing or invalid values produce a validation response. Saving a manual version does not establish platform acceptance.

Backend interfaces

The /api/v1 endpoints cover problems, due queues, topics, version lists and commits, explicit review completion, JSON ingestion, statistics, heatmaps, and backup export/restore. FastAPI provides interactive documentation at /docs. The interface and API use the same local origin.

Security and user management

Descriptions are sanitized with Bleach, problem links accept HTTP/HTTPS, and SQL values are parameterized. Uploads are limited to 25 MB and batches to 10,000 records. The server binds to loopback. This release has no authentication or multi-user isolation; it is intended for one local user.

6 Testing results and achieved outcome

The automated suite was rerun on 15 October 2026. All 13 tests passed in approximately 3 seconds. Tests use temporary databases. The real export and read-only tracker are used for integration checks when those files are available.

Test scenario	Verified behavior	Result
Real batch and repeat import	274 exact code snapshots; repeat import skips all records.	Pass
Original tracker migration	Original IDs, labels, dates, stages, and history retained.	Pass
GFG export variants	Nested and universal formats; distinct submission IDs retained.	Pass
Invalid records and HTML	Invalid JSON reported; executable HTML removed.	Pass
Review cycle and replay	7/30/180-day intervals; stale completion rejected.	Pass
Concurrent review requests	Exactly one request advances the stage.	Pass
Version persistence	Manual code survives restart without a review change.	Pass
Backup round trip	Versions and reviews restored with remapped parent IDs.	Pass
Invalid backup	Orphan references rejected without partial changes.	Pass
Import write failure	Simulated failure rolls back the entire valid import.	Pass
Manual entries and topics	Required values validated; future reviews rejected.	Pass
Identity and URL checks	Platform mismatch and unsafe manual link rejected.	Pass
Existing activity enrichment	Import enrichment preserves manual addition activity.	Pass

Browser verification

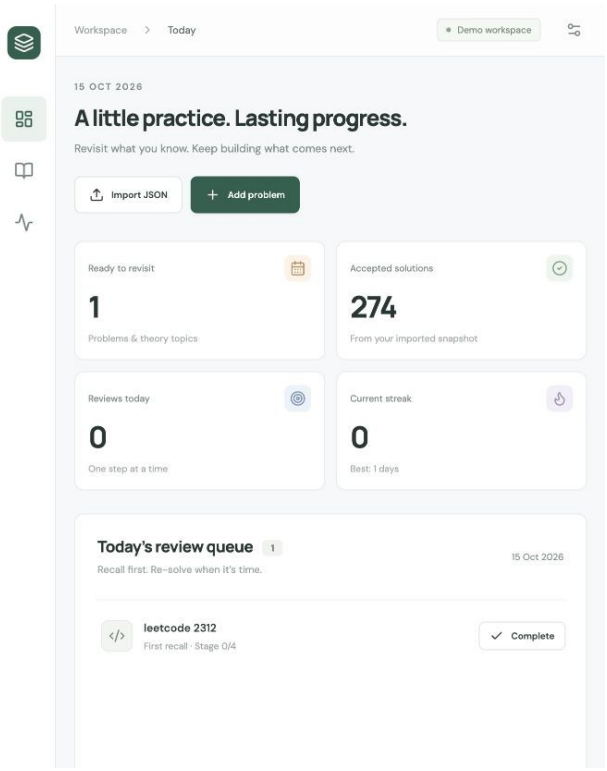
Browser checks confirmed search, imported descriptions and code, manual version saving, explicit due-review completion, repeat upload of the 274-record batch, a real GFG import, backup download and additive restore. Restarting the server preserved the saved version and completed review. The frontend TypeScript build passed.

Achieved outcome

The initial library contains 275 problems: 274 accepted solutions from the verified export and one preserved unverified tracker record. The export contains 143 Easy, 98 Medium, and 33 Hard problems, with 269 C++, one C, and four MySQL solutions. The additional tracker record is Medium. One theory topic and its recorded review are retained. These counts describe the imported snapshot, not a live lifetime solve total.

7 Dashboard and searchable library

The dashboard separates due reviews, accepted solutions, completed reviews, and streaks. The library distinguishes accepted and unverified records and provides filters for locating a stored solution. Screenshots show the responsive layout used in the demonstration browser.



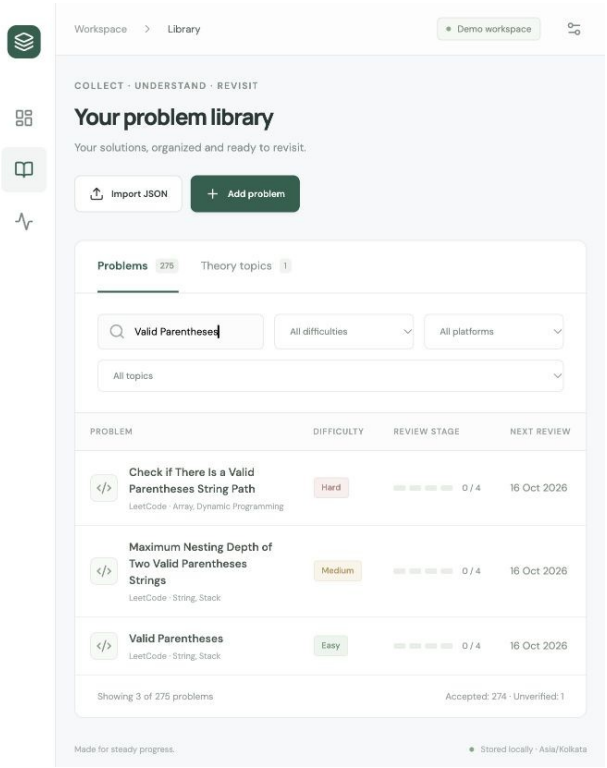
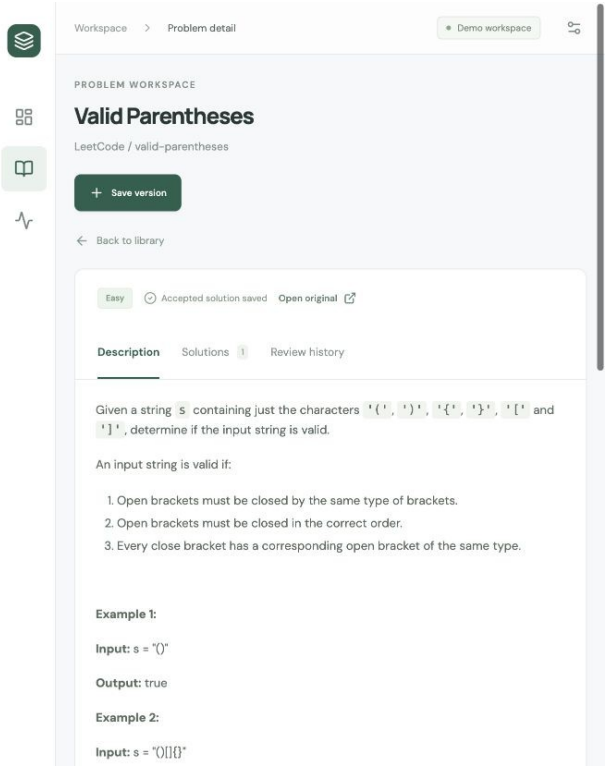


Figure 2 Today dashboard with one due item and 274 accepted solutions; Figure 3 Library filtered to Valid Parentheses with difficulty platform and topic controls.

8 Problem description and saved code

Problem Detail provides an offline question reader and version selector. The accepted snapshot retains exact code from the export. Import dates are shown separately from solve dates, which remain unavailable when the export contains no solve timestamp.



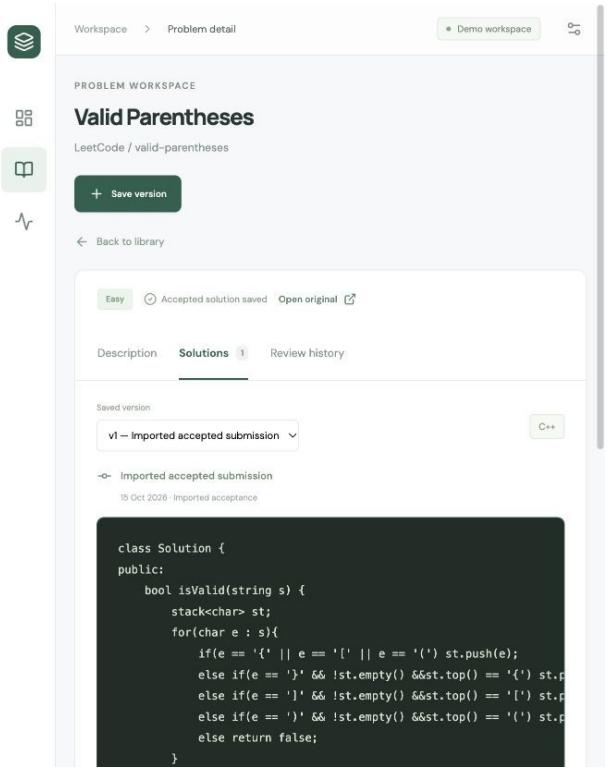
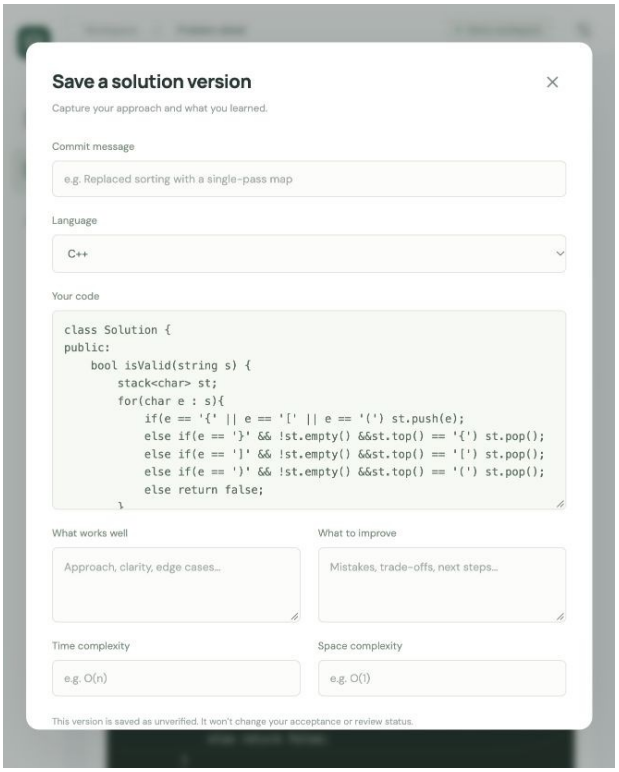


Figure 4 Sanitized Valid Parentheses description and examples; Figure 5 Exact imported C++ code and its initial solution version.

9 Solution form and progress view

The version form accepts code, language, a commit message, self-assessment, and complexity notes. The progress view summarizes the collection and actual recorded activity. Historical batch imports do not create synthetic solve events or review activity.



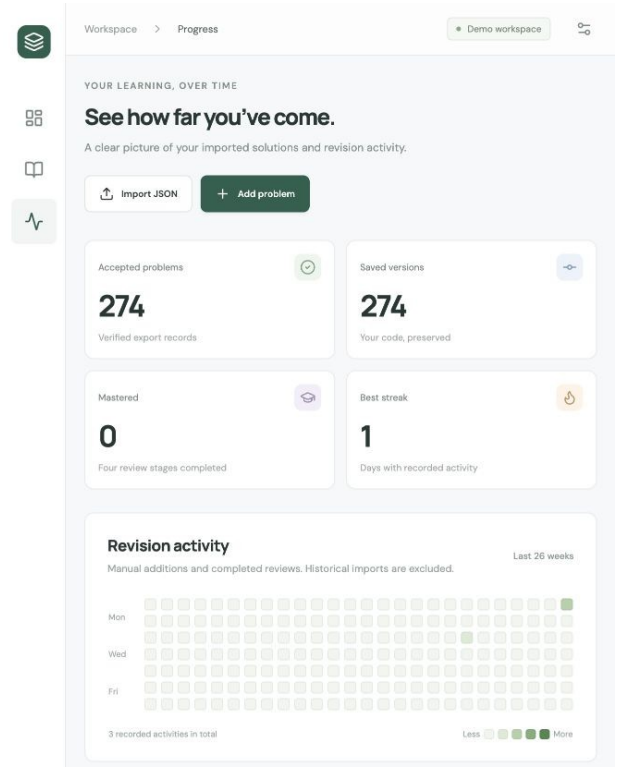


Figure 6 Form for saving a manual version with message and assessment; Figure 7 Collection statistics and revision activity heatmap.

10 Running the project and conclusion

Setup and demonstration

The project requires Python 3.11 or later and a compatible Node.js installation. Run ./start.sh from the project directory. It installs missing dependencies, builds the frontend, prepares data, and serves the demo at http://127.0.0.1:3000. Use PORT=3001 if port 3000 is occupied.

On another computer, the application can start with an empty database. Import the submission JSON through the interface or provide explicit paths to the preparation script. The original personal export is not required to start the application. Dependency lockfiles support reproducible installation.

- Show the imported library and open Valid Parentheses to display real saved code.
- Save a manual version with a message and self-assessment.
- Complete the preserved overdue tracker problem and inspect the new review date.
- Restart the server to demonstrate persistence, then download a full backup.

Viva preparation

Explain the difference between request schemas and database tables; how routes call services; why prepared SQL parameters are used; how a transaction prevents partial writes; how platform plus slug identifies a problem; and how expected_stage rejects repeated review requests. Explain that acceptance and revision status represent different facts.

For Django comparisons, a Django view handles a request as a FastAPI route does here. Django templates render server-side HTML, while this project uses a React interface. Django ORM models represent database records; this implementation uses SQL tables and queries. Pydantic request schemas validate API input rather than acting as Django ORM models. No Django implementation is claimed.

Limitations and future work

The current release does not judge or execute saved code, authenticate users, synchronize across devices, or capture live extension submissions. Future work includes queued extension delivery, version comparison, draft solution imports, user accounts where needed, and summary/activity APIs for a broader personal planning application. AI feedback would remain advisory and separate from student notes.

Conclusion

AlgoFlow provides a working local flow from exported accepted solutions to searchable code, deliberate revision, and recorded solution versions. Real-data integration, preserved tracker history, duplicate protection, restart persistence, and backup restoration are verified. The release establishes a practical foundation for learning backend development and extending the personal DSA workflow.

Submission materials

The submission package consists of the one-page summary, this report, and the GitHub source repository. README.md documents startup and architecture; DEMO.md provides the presentation walkthrough; the tests and dependency lockfiles support verification and reproduction.

GitHub repository https://github.com/KRISH141006/AlgoFlow